

Project Proposal Predicting One Piece Card Game Leader Matchups

Kiana Kiser | HS26 | I.BA_MLOPS | Github Repository: github.com/kianakiser/optcg-swiss-forecast

1 Problem statement

What I want to predict: One Piece Card Game is a competitive trading card game in which two players use a Leader and a 50-card deck. The Leader shapes how the deck is built and played. I want to estimate the probability that a player using Leader A wins against a player using Leader B. The forecast describes the deck builds and players represented in the data, rather than a specific deck.

Horizon and scope: I make the forecast once the pairing and Leaders are known, before the round begins, with a match horizon of 60 minutes. I include decided Swiss-round matches from Limitless tournaments with at least 32 players, from September 2024 onward. Swiss rounds pair players with similar records without eliminating them after a loss. I exclude top cut, byes, draws and matches without a recorded winner.

Who it is for: Players choosing a Leader who want to compare recent tournament performance and the chance of winning specific matchups. The page will show recent win rates and match counts alongside the prediction, rather than claim one Leader is strongest against every opponent.

How I will judge it: I use the Brier score: the squared difference between the predicted probability and the result (1 for a win, 0 for a loss), averaged over test matches. Lower is better; always predicting 50% scores 0.25. My target is below 0.25 and below a historical Leader-matchup lookup on the same later tournaments. I will report the results even if ML does not improve on that baseline.

2 Originality and motivation

Why I chose this: I have played One Piece for about a year and compete in tournaments. I already collect results for a personal side project, so I have roughly two years of history to start from. I want a clearer view of matchups than a single overall win rate gives me. Different deck builds, card interactions, player decisions and luck make this uncertain, which is why I want to estimate probabilities rather than promise a winner.

What is different: I checked past HSLU MLOps projects and the linked KTH examples. They include sports and chess predictions, so predicting a winner is not new. My project applies this to One Piece Leader matchups. New cards can change the decks played under the same Leader, making fresh results and regular retraining important.

3 Data source and features

Source and updates: Limitless TCG publishes tournament results through a public JSON API. It requires no login and does not need scraping. My personal collection has about 68,000 decided Swiss matches from 252 tournaments between September 2024 and September 2026. Recent growth is about 590 matches a week. The new course pipeline will backfill that history, then check for finished tournaments daily. I plan to retrain weekly and will confirm the terms for ongoing API use with Limitless.

Features: The two Leader IDs are known before play. Historical features use only tournaments from earlier dates:

- Each Leader's earlier win rate and the number of matches behind it.
- The earlier win rate for this exact Leader pairing and its sample size.
- Each Leader's win rate over the preceding 28 days and the number of matches behind it.

Label and evaluation. The label is the winner recorded in the pairing data; it cannot be calculated from these inputs. The first listed player won about 50.2% of the collected decided matches, so there is no rare positive class. I will discard outcome fields such as final placing at ingestion. I will compute history only from earlier dates, including treating events on the same date as simultaneous, and test on later events rather than random matches.

4 System design

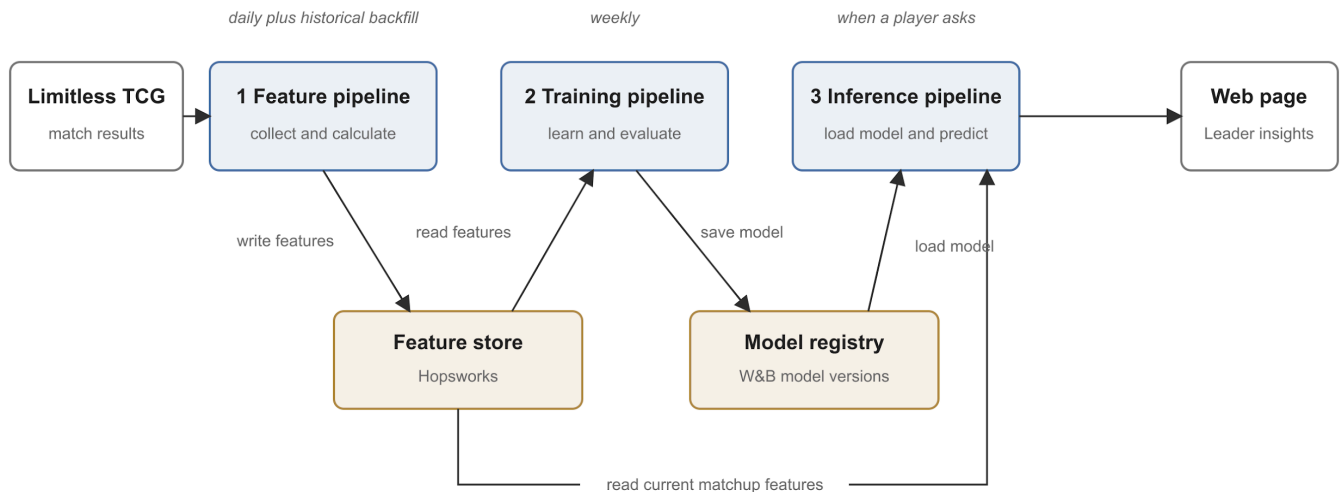


Figure 1 The feature, training and inference pipelines share a feature store and model registry

Core workflow: The daily feature pipeline will collect results, calculate historical rates and save them. The weekly training pipeline will train a scikit-learn gradient boosting model, test it on later tournaments and register a model version with its scores. When a player asks, the inference pipeline will load the selected model and current matchup features, then return a probability, recent win rates and match counts. I will also log failed jobs, the deployed model version and changes in Leader usage. The pipelines run separately through shared storage.

Tools: Hopsworks will store features for training and prediction. Weights & Biases will track training runs and keep model versions in its registry. Both appear in the module materials. GitHub Actions will schedule jobs and tests. FastAPI will provide the prediction endpoint, packaged with Docker and hosted on Google Cloud Run. Python and scikit-learn keep modelling manageable; pytest checks the calculations, and uv pins dependencies so the environment can be reproduced.

Optional work: If the core pipelines work reliably, I may test whether full decklists improve predictions where both submitted lists are available. A more detailed monitoring dashboard is optional.